

End-to-End Time Series Forecasting System with API

This project focuses on building a production-ready time series forecasting system to predict the next 8 weeks of sales for different states using historical sales data. The system was designed to simulate a real-world backend forecasting service capable of training multiple machine learning and deep learning models, comparing their performance, selecting the best model automatically, and serving predictions using a REST API.

1. Objective

The primary objective of this project was to create an automated forecasting pipeline that handles missing values, captures seasonality and trends, implements multiple forecasting algorithms, automatically selects the best-performing model, and exposes predictions through an API.

2. Dataset Overview

The dataset was provided in Excel format and contained historical sales information for multiple states. Key columns included Date, State, and Sales.

3. Data Preprocessing

Several preprocessing steps were applied including datetime conversion, sorting by state and date, handling missing dates using resampling, and filling missing sales values using forward fill techniques.

4. Feature Engineering

Important features such as lag features ($t-1$, $t-7$, $t-30$), rolling mean, rolling standard deviation, day of week, and month were created to improve forecasting performance.

5. Models Implemented

The project implemented ARIMA/SARIMA, Facebook Prophet, XGBoost, and LSTM models to compare both traditional statistical forecasting methods and modern machine learning/deep learning approaches.

6. Model Evaluation

Time-series-based train-validation splitting was used to avoid data leakage. Mean Absolute Error (MAE) was used as the evaluation metric for model comparison.

7. API Development

A REST API was developed using FastAPI. The API loads the trained model and provides prediction endpoints for forecasting future sales values.

8. Project Architecture

The project follows a modular structure with separate files for preprocessing, feature engineering, model training, evaluation, and API deployment.

9. Challenges Faced

Major challenges included handling missing dates, generating meaningful time-series features, managing time-based validation, and comparing multiple forecasting models effectively.

10. Conclusion

The project successfully demonstrates a scalable and production-ready forecasting system capable of automated model selection and API-based deployment for real-world business use cases.

11. Technology Stack

Technology	Purpose
Python	Programming Language
Pandas & NumPy	Data Processing
XGBoost	Machine Learning
Prophet	Forecasting
TensorFlow/Keras	LSTM Model
FastAPI	REST API
Statsmodels	ARIMA/SARIMA

Prepared By: Shefali Jakale
Project: Data Science Case Study